

Herramientas Computacionales Avanzadas

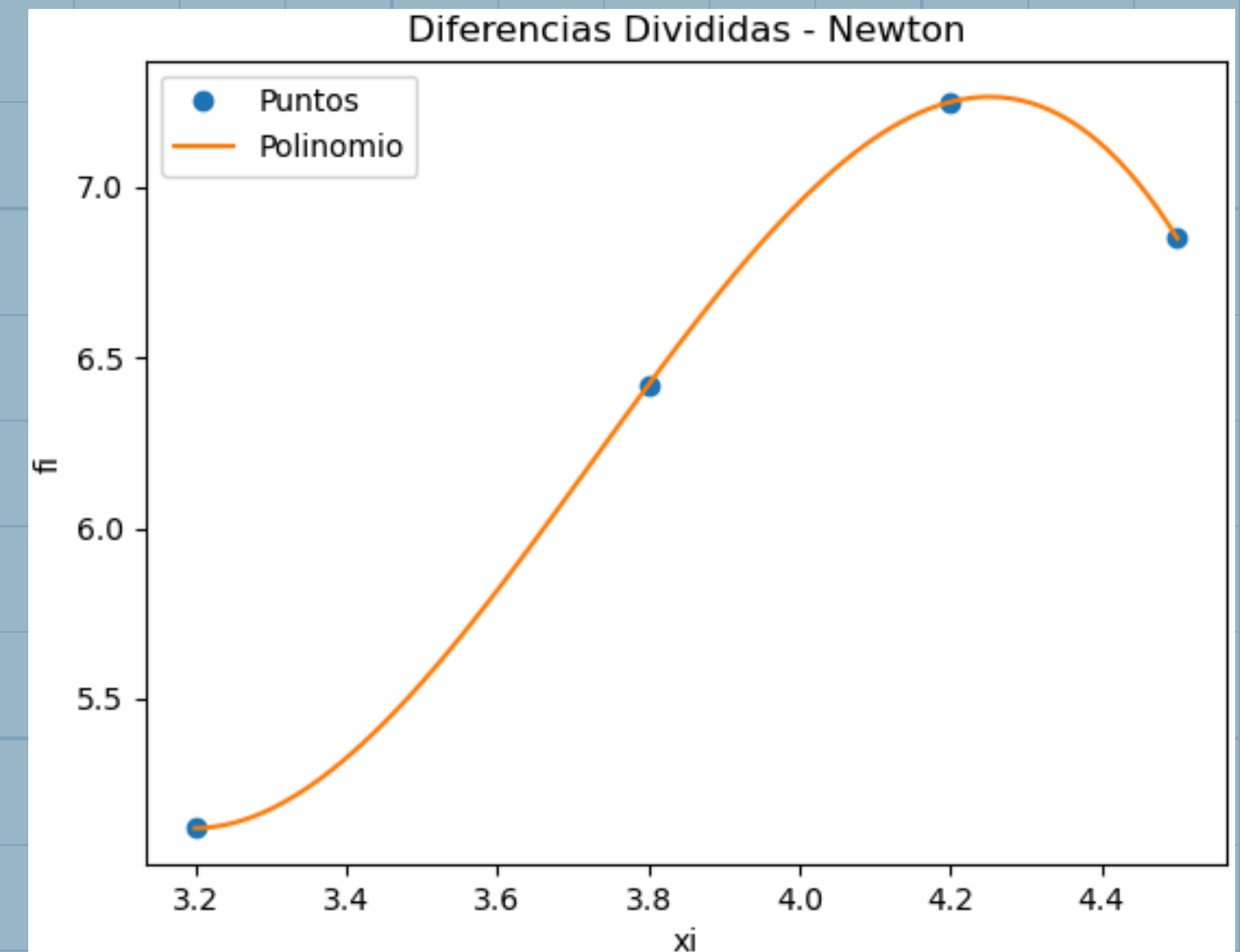
INTERPOLACIÓN DE NEWTON

Equipo. Paleochaparritas

FUNDAMENTOS MATEMÁTICOS

La interpolación es un procedimiento numérico en el que se estiman valores intermedios entre datos definidos por puntos. [Chapra S. C. y Canale, R. P. 2007, pag 503]

En la interpolación de Newton se toma en cuenta la distancia y el ritmo de cambio entre cada par de puntos. A través de las diferencias divididas, se calcula qué tan rápido cambia el valor entre una coordenada y la siguiente, logrando que la curva se doble de forma suave y natural para conectar todo.



Interpolación de Newton. Fuente: blog.espol.edu.ec

POLINOMIO GENERAL

1

Definir el conjunto de puntos reales (x, y) .

2

Construir la tabla escalonada calculando las tasas de cambio entre los puntos. De aquí se extraen los coeficientes.

$$f_n(x) = b_0 + b_1(x-x_0) + \cdots + b_n(x-x_0)(x-x_1) \cdots (x-x_{n-1})$$

los coeficientes $b_0, b_1, \dots, b_n$ son

$$b_0 = f(x_0)$$

$$b_1 = f[x_1, x_0]$$

$$b_2 = f[x_2, x_1, x_0]$$

3

Sustituir los coeficientes calculados en la fórmula para obtener la ecuación exacta que dibuja la curva.

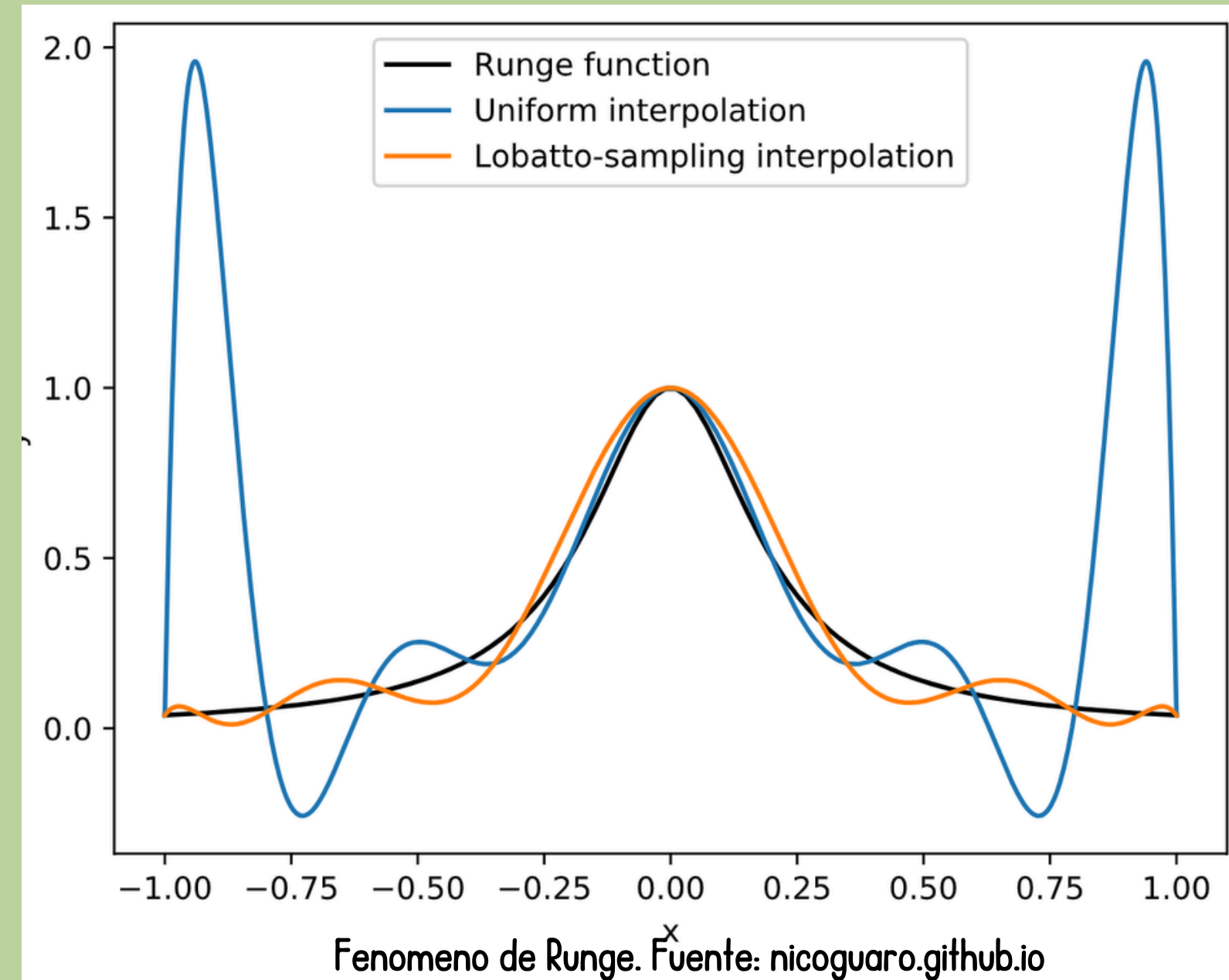
4

Sustituir el valor de x que queremos investigar dentro de nuestra nueva ecuación para descubrir su resultado.



¿EXISTE UNA LIBRERÍA EN PYTHON?

Intentar unir muchos datos con un solo polinomio gigante causa vibraciones irreales en los extremos de la curva (Fenómeno de Runge). Además que el proceso es manual y existen librerías como SciPy que prefieren utilizar métodos modernos por fragmentos.



¿ENTONCES?

- Para implementarlo, necesitamos construir una función propia en Python que utilice ciclos for anidados y matrices vacías (`np.zeros`).
- El código debe hacer dos cosas: primero, calcular nuestra propia "tabla de diferencias divididas" restando los datos iterativamente, y segundo, usar esos resultados para armar matemáticamente la ecuación final.

CÓDIGO

```
import numpy as np
import matplotlib.pyplot as plt

def InterpolNewton(x, y, xi):
    n = len(x)

    dd = np.zeros([n, n])
    b = []

    for i in range(n):
        for j in range(n):
            if j == 0:
                dd[i, j] = y[i]
            elif i < j:
                dd[i, j] = dd[i, j]
            else:
                dd[i, j] = (dd[i, j-1] - dd[i-1, j-1]) / (x[i] - x[i-j])

            if i == j:
                b.append(dd[i, j])

    xterm = []
    pol = []
```

```

    for i in range(n):
        if i == 0:
            xterm.append(1)
            pol.append(b[i] * xterm[i])
        else:
            xterm.append(xterm[i-1] * (xi - x[i-1]))
            pol.append(pol[i-1] + b[i] * xterm[i])

    return x, dd, b, xterm, pol

x = [50, 100, 200, 300]
y = [25.5, 32.1, 48.0, 65.3]
xi = 150

resultados = InterpolNewton(x, y, xi)

b = resultados[2]
pol = resultados[4]

xpol = np.linspace(min(x), max(x), 50)
n = len(x) - 1

ypol = np.zeros(len(xpol))

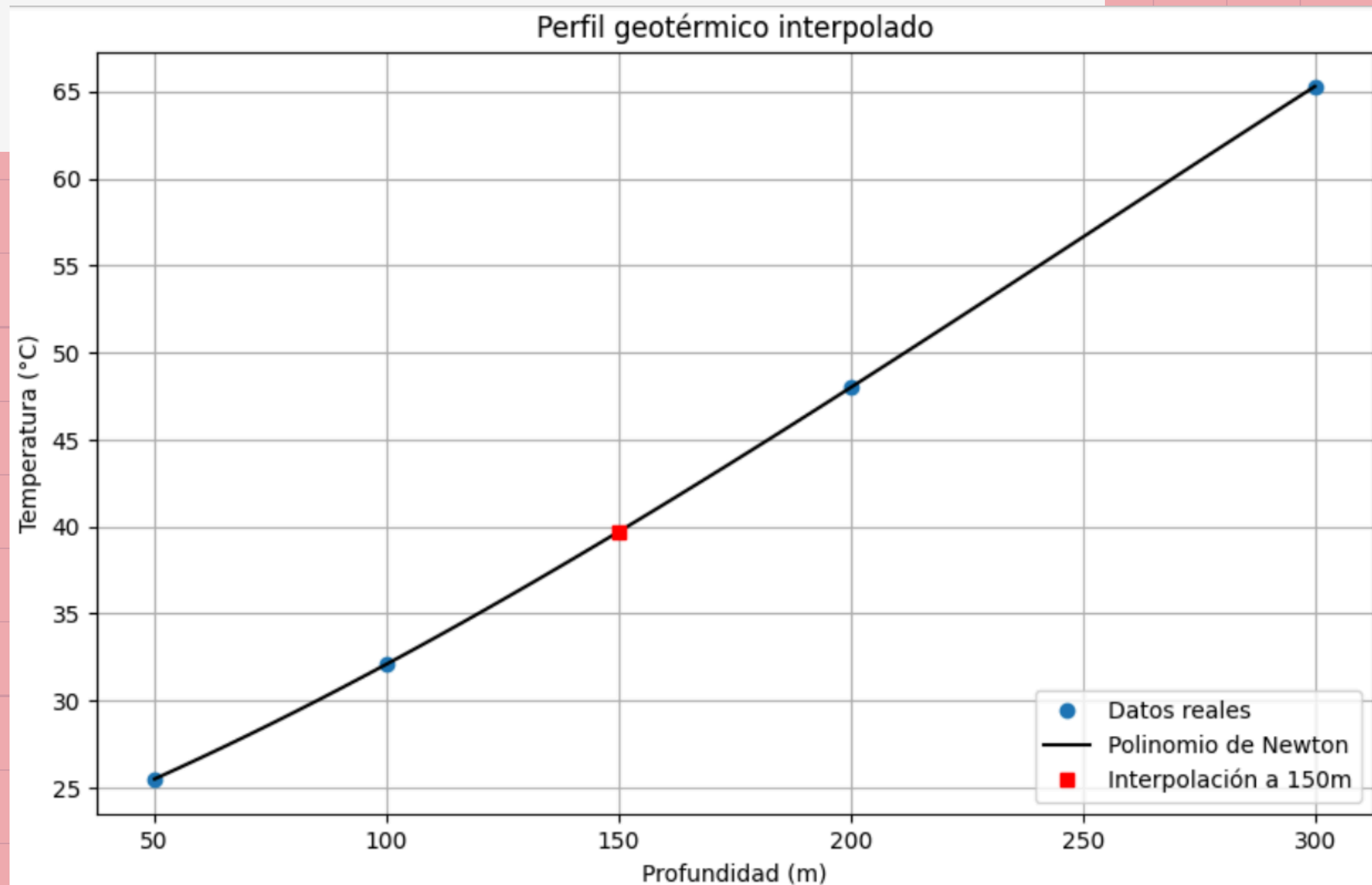
for i in range(len(xpol)):
    for j in range(n+1):
        if j == 0:
            xterm_plot = 1
            ypol[i] = b[0]
        else:
            xterm_plot = xterm_plot * (xpol[i] - x[j-1])
            ypol[i] = ypol[i] + b[j] * xterm_plot

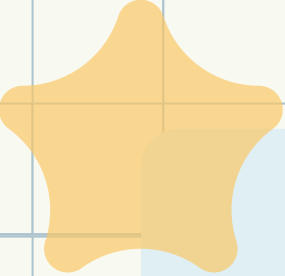
```



```
plt.figure(figsize=(10,6))
plt.plot(x, y, "o", label='Datos reales')
plt.plot(xpol, ypol, '-', c='k', label='Polinomio de Newton')
plt.plot(xi, pol[-1], "sr", label='Interpolación a 150m')
```

```
plt.xlabel("Profundidad (m)")
plt.ylabel("Temperatura (°C)")
plt.title("Perfil geotérmico interpolado")
plt.grid(True)
plt.legend(loc=4)
```





REFERENCIAS

- Chapra, S. C., & Canale, R. P. (2007). Métodos numéricos para ingenieros (5.^a ed.). McGraw-Hill Interamericana.
- ingangelocastano. [s.f.]. 4. Interpolacion Metodo de Newton.ipynb [Código fuente]. GitHub Gist. Recuperado de <https://gist.github.com/ingangelocastano/d8383d298890b8aae2af1b51217d1c6b>

